Module 2:
Variables and Expressions

Sarah Foss

Some slides adapted for use from Dr. Ramon Lawrence

Objectives

- To use variables to store data
- To explore Java numeric primitive data types: byte, short, int, long, float, and double
- To obtain input from the console using the Scanner class
- To perform operations using operators +, -, *, /, and %
- To cast the value of one type to another type

Translating An Algorithm

Computing the Area of a Pizza

Algorithm in English:

1. Read in the pizza's radius.
2. Calculate the pizza area using:

   $$pizzaArea = radius \times radius \times \pi$$

   (π is important for pizza pies too!)
3. Display the size of your pizza (so you know how much pizza you'll be enjoying).

ComputePizzaArea

```
public class PizzaAreaCalculator {
    public static void main(String[] args) {
        // Declare variables for storing results
        double pizzaArea;
        double radius;

        // Create a scanner object to read input
        Scanner scanner = new Scanner(System.in);

        // Step 1: Read in the pizza's radius
        System.out.print("Enter the radius of the pizza: ");
        radius = scanner.nextDouble();

        // Step 2: Compute the pizza area using the formula
        pizzaArea = radius * radius * 3.14159;

        // Step 3: Display the result
        System.out.println("The area of your pizza is: " + area + " square units.");
    }
}
```

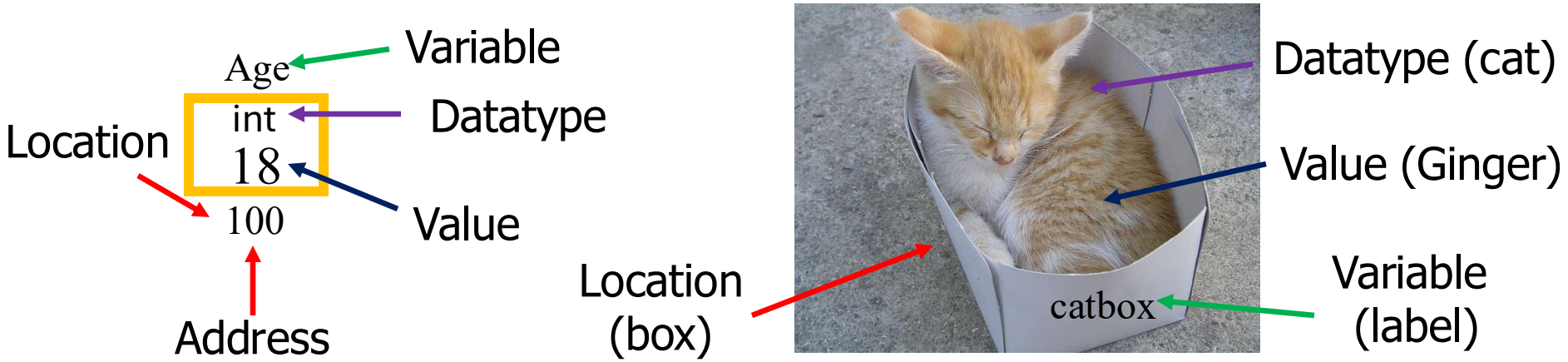
Values, Variables, Datatypes, and Locations

A **value** is a data item that is manipulated by the computer.

A **variable** or **identifier** is the name that the programmer uses to refer to a location in memory.

A **datatype** is the type of data stored at a location in memory.

A **location** has an address in memory and stores a value.



IMPORTANT: The value at a given location in memory (named using a variable name) can change using initialization or assignment.

Declaring Variables

To create a variable, you must specify the datatype and the identifier:

Syntax:

```
Datatype identifier;
```

Examples:

```
int x;          // Declare x to be an
                // integer variable;
double radius; // Declare radius to
                // be a double variable;
char a;         // Declare a to be a
                // character variable;
```

This tells the compiler to allocate appropriate memory space based on the datatype.

Variable Rules

Variables are also called identifiers. An ***identifier*** is a name that cannot begin with a number and cannot contain spaces.

Variables:

- must be declared.
- can be declared anywhere in the program, but usually should be declared at the start of the code.
- **ARE** case-sensitive. Numbers are allowed (but not at the start). Only other symbols allowed are the underscore ('_') and the dollar sign ('$').
- cannot be Java keywords (e.g., class, int, public, etc.).

Warning!

```
double interestRate = 0.045;
double interest = interestrate * 35;
```

Assignment Statements

An **assignment statement** changes the value of a variable.

- The variable on the left-hand side of the ***assignment operator*** (=) is assigned the value from the right-hand side.
- The value may be changed to a constant, to the result of an expression, or to be the same as another variable.
- The values of any variables used in the expression are always their values before the start of the execution of the assignment.

Syntax:

```
variable = expression;
```

```
int a;
int b;
a = 5;
b = 10;
a = 10 + 6 / 2;
b = a;
a = 2 * b + a – 5;
```

Declaring and Initializing in One Step

A variable must be declared before it can be assigned a value. Whenever possible declare a variable and assign its initial value (**initialization**) in one step.

Example:

```
int x = 1;
```

is equivalent to

```
int x;
x = 1;
```

Naming Conventions

Choose meaningful and descriptive names.

Use camel casing for variable and method names:

camelCase: the first word is written in all lowercase, and each subsequent word begins with an uppercase letter, without spaces or underscores between them.

```
int radius;
double pizzaArea;
```

Named Constants

A variable's value can change while a program is running, but a **named constant** holds data that remains fixed and does not change throughout the program's execution.

Syntax:

```
final datatype CONSTANT_NAME = VALUE;
```

```
final double PI = 3.14159;
final int MAX_PIZZA_SIZE = 15;
```

Also referred to as a **final variable** or simply a **constant**.

A constant should be declared and initialized in the same statement.

SNAKE_CASE: Constant identifiers should be written in uppercase with underscores to connect words.

Numeric Datatypes

A **primitive datatype** is a basic type of data that is built into the language. It represents simple values, like numbers or characters.

The following are the primitive numeric types built into Java:

Type	Size	Range
Integers (whole numbers)		
byte	1 byte	-128 to 127
short	2 bytes	-32,768 to 32,767
int	4 bytes	-2,147,483,648 to 2,147,483,647
long	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
Floating point numbers (fractional numbers)		
float	4 bytes	Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Sufficient for storing 15 to 16 decimal digits

Numeric Literals

A **numeric literal** is a direct representation of a number within the code. It is a fixed value that you write directly in your program, which the compiler interprets as a specific numeric type (such as int, double, float, etc.).

Integer literals are by default int, , but you can specify a long by appending l or L.

```
int num = 100;          // Integer literal, type is int
long bigNum = 100L;     // Long integer literal, type is long
```

Floating-point literals are by default double, but you can specify a float by appending f or F.

```
double pi = 3.14159;    // Double literal, type is double
float euler = 2.718f;   // Float literal, type is float
```

Integer Overflow

Integer overflow happens when a calculation produces a number that is too big or too small to fit in the storage space of an integer type (like int or byte).

Each integer type has a limited range, and when you exceed that range, the number "wraps around" to the other side.

```
int value = 2147483647 + 1;
// value will actually be -2147483648
```

Double vs Float

Because double has more memory (twice the size of float), it can store numbers with much greater precision, meaning it can handle more digits after the decimal point without losing accuracy.

```
float floatValue = 3.141592653589793238f;
// Only accurate to ~7 decimal places
double doubleValue = 3.141592653589793238;
// Accurate to ~16 decimal places

System.out.println("Float value: " + floatValue);
// Output: 3.1415927
System.out.println("Double value: " + doubleValue);
// Output: 3.141592653589793
```

Calculating with Floating Point Numbers

Calculations with floating-point numbers aren't perfectly accurate because computers can't store them exactly. Some decimal values can't be represented in binary, which leads to small rounding errors. This makes floating-point calculations only approximate, especially for very large or small numbers.

For example,

```
System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);

//displays 0.5000000000000001, not 0.5
```

```
System.out.println(1.0 - 0.9);

//displays 0.09999999999999998, not 0.1
```

Using integers instead of floating-point numbers is a common approach when precision is critical in certain calculations because integers are stored exactly.

Reading Input from the Console

```
// Step 1: import java.util package (or  import java.*)
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
        // Step 2: Create a Scanner object to read input
        Scanner scanner = new Scanner(System.in);

        // Step 3: Reading different types of input
        // Reading a string
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();

        // Reading an integer
        System.out.print("Enter your age: ");
        int age = scanner.nextInt();

        // Reading a double
        System.out.print("Enter your height in meters: ");
        double height = scanner.nextDouble();

        // Step 3: Display the inputs
        System.out.println("Name: " + name + "Age: " + age  + "Height: " + height);
    }
}
```

Expressions

An expression is a sequence of operands and operators that yield a result. An expression contains:

- **operands** - the data items being manipulated in the calculation

  `5, "Hello, World", myDouble`

- **operators** - the operations performed on the operands

  `+, -, /, *, %`

An operator can be:

- **unary** - applies to only one operand

  ```
  d = -3.5;          // "-" is a unary operator, 3.5 is the operand
  ```

- **binary** - applies to two operands

  ```
  d = e * 5.0;       // "*" is binary operator, e and 5.0 are operands
  ```

Operators

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Remainder	Returns the division remainder	x % y

Division

When you divide two integers, the result will be an integer. Any fractional part (remainder) is discarded, meaning the result is truncated toward zero.

5 / 2 yields an integer 2

When dividing two floating-point numbers, the result will be a floating-point number, and the decimal part will be preserved.

5.0 / 2.0 yields a double value 2.5

If you divide an integer by a floating-point number, Java converts the integer into a floating-point number, and the result will be a floating-point number, and the decimal part will be preserved.

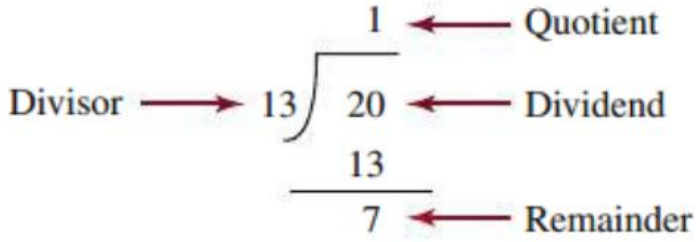
5.0 / 2 yields a double value 2.5

Remainder Operator

The remainder operator (%) yields the remainder after division.

Example:

20 % 13 is 7



Saturday is the 6th day in a week

A week has 7 days

(6 + 10) % 7 is 2

The 2nd day in a week is Tuesday

After 10 days

Displaying Time

```
import java.util.Scanner;

public class DisplayTime {
  public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    // Prompt the user for input
    System.out.print("Enter time in seconds: ");
    int seconds = input.nextInt();

    int minutes = seconds / 60;
    int remainingSeconds = seconds % 60;
    System.out.println(seconds + " seconds is " + minutes +
      " minutes and " + remainingSeconds + " seconds");
  }
}
```

Exponents

You can calculate exponents using the Math.pow() method, which is part of the java.lang.Math class.

The Math.pow() method takes two arguments:

1. The base (the number you want to raise to a power)
2. The exponent (the power to which the base should be raised)

```
double base = 2.0;
double exponent = 3.0;
double result = Math.pow(base, exponent);  // 2 raised to the power of 3

System.out.println("Result: " + result);   // Outputs: 8.0
```

Expressions - Operator Precedence

Each operator has its own priority similar to their priority in regular math expressions:

1. Any expression in parentheses is evaluated first starting with the inner most nesting of parentheses.
2. Unary + and unary - have the next highest priorities.
3. Multiplication and division (*, /, %) are next.
4. Addition and subtraction (+,-) are then evaluated.

Example:

20 – ((4 + 5) - (3 * (6 - 2))) * 4 = ?

20 – ((4+5) – (3*4)) *4
20 – (9 - 12) * 4
20 – (- 3) * 4
20 – (-12)
20 + 12 **= 32**

Augmented Assignment Operators

Operator	Example	Same As
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3

Increment and Decrement

The **increment** (++) and **decrement** (--) operators are used to increase or decrease a variable's value by 1.

They can be written in two ways: prefix and postfix, which affect when the change happens.

- **Increment Operator (++):**
  - Prefix (++variable): The value is increased before it's used in the expression.
  - Postfix (variable++): The value is used first, then increased after.

```
int x = 5;
int y = ++x;   // x is increased to 6, then y gets the value 6
int z = x++;   // z gets the value 6, then x increases to 7
```

- **Decrement Operator (--):**
  - Prefix (--variable): The value is decreased before it's used.
  - Postfix (variable--): The value is used first, then decreased after.

```
int a = 5;
int b = --a;   // a decreases to 4, then b gets the value 4
int c = a--;   // c gets the value 4, then a decreases to 3
```

Warning!

Using the increment (++) and decrement (--) operators too much, especially in complicated expressions, can make your code confusing and harder to understand.

This can cause mistakes that are hard to notice.

It’s a good idea to use them carefully and keep your code simple by breaking complex operations into smaller, clearer steps.

Clear code is better than code that is just short.

Numeric Conversion

Can you use two different types in a calculation?

Yes. If an integer and a floating-point number are used together, Java automatically changes the integer to a floating-point number.

For example,

`2 * 3.5` becomes `2.0 * 3.5`

When two different types are used in a calculation:

1. If one is a `double`, the other becomes a `double`.
2. If not, and one is a `float`, the other becomes a `float`.
3. If not, and one is a `long`, the other becomes a `long`.
4. If none of the above, both are changed to `int`.

Casting

Casting is the process of converting one numeric type to another.

There are two types of casting: implicit (automatic) and explicit (manual).

Implicit Casting (Widening Conversion): The compiler automatically converts smaller numeric types to larger ones when there's no risk of losing information.

```
double d = 3; // d holds 3.0
```

Explicit Casting (Narrowing Conversion): When converting from a larger type to a smaller type (e.g., from double to int), you must explicitly tell the compiler to perform the conversion. This is called narrowing, and it can result in the loss of information (like cutting off decimal points).

```
int i = (int)4.99;  // i holds 4
```

What is wrong? `int x = 7 / 2.0;`

Summary

- **Identifiers** are names used for elements like variables, methods, and classes. They can include letters, digits, underscores, and dollar signs, but must start with a letter or underscore, not a digit. Identifiers can't be reserved words and can be any length.
- **Variables** store data in a program. Declaring a variable tells the compiler what type of data it will hold.
- **Import statements** come in two types: specific import (imports a single class) and wildcard import (imports all classes in a package).
- In Java, the **equal sign (=)** is used to assign values to variables. A variable declared in a method must be assigned a value before it can be used.
- A **named constant** is a value that doesn't change and is declared using the final keyword.
- Java provides four **integer types** (byte, short, int, long) for integers of different sizes.
- Java has two **floating-point types** (float and double) for numbers with decimal points of different precision.

Summary

- Java supports basic **arithmetic operators**: + (addition), - (subtraction), * (multiplication), / (division), and % (remainder).
- Integer division gives integer results.
- **Augmented assignment operators** include += (add and assign), -= (subtract and assign), *= (multiply and assign), /= (divide and assign), and %= (remainder and assign).
- The **increment operator (++)** increases a variable's value by 1, and the **decrement operator (--)** decreases it by 1.
- When mixing types in an expression, Java automatically converts values to the appropriate type.
- **Type casting** allows converting from one type to another. **Widening** (small to large type) happens automatically, while **narrowing** (large to small type) requires explicit casting.